УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА

ОСНОВЕ ИНФОРМАЦИОНЕ БЕЗБЕДНОСТИ

ПРАКТИКУМ

Нови Сад

Предговор

Практикум је самостална академска наставна публикација из основа информационе безбедности. Циљ није меморисање назива контрола, већ разумевање зашто се одређена контрола уводи, која се претња њоме ублажава и којим се доказом потврђује њено исправно понашање. Градиво почиње идентитетом и приступом, а завршава се корелацијом догађаја и проценом ефективности контрола.

Осам поглавља прати развојни пут од идентитета, аутентикације и ауторизације, преко конфигурације, модела претњи и оперативне безбедности, до контекстуалног приступа и мерења ефективности контрола. Свака тема повезана је са стварним C#/.NET наставним примером и аутоматизованим тестовима који показују и дозвољено и недозвољено понашање.

Сви примери се извршавају искључиво у контролисаном наставном окружењу. Сценарији злоупотребе служе анализи и дефанзивном тестирању и не подразумевају рад над туђим системима. Терминологија и код у тексту задржавају оригиналне називе технологија и програмских идентификатора.

Садржај

Предговор ..... —
Како користити практикум 1
1 Идентитет, аутентикација и контрола приступа заснована на улогама 2
1.1 Ментални модел приступа 2
1.2 Примена принципа и честе грешке 3
1.3 Пример: оператор добија нову одговорност 3
1.4 Дизајнерска одлука: улога није дозвола 3
1.5 Како знамо да модел приступа ради 4
1.9 Програмски пример и самостални задатак 6
2 Ауторизација над ресурсом и класификација података 7
2.1 Одлука мора имати контекст 7
2.2 Примена принципа и честе грешке 8
2.3 Пример: исправна улога, погрешан запис 8
2.4 Дизајнерска одлука: податак носи обавезе 8
2.5 Како знамо да је ауторизација потпуна 8
2.9 Програмски пример и самостални задатак 10
3 Политике, конфигурација и безбедносна видљивост 12
3.1 Правило без доказа је само намера 12
3.2 Примена принципа и честе грешке 13
3.3 Пример: промена правила без губитка историје 13
3.4 Дизајнерска одлука: правило мора имати границе 13
3.5 Видљивост као способност тима 14
3.9 Програмски пример и самостални задатак 16
4 Имовина, границе поверења и моделовање претњи 17
4.1 Модел пре контроле 17
4.2 Примена принципа и честе грешке 17
4.3 Пример: извоз из интерног система 18
4.4 Дизајнерска одлука: прво модел, затим алат 18
4.5 Одржавање модела претњи 18
4.9 Програмски пример и самостални задатак 20
5 Вишефакторска аутентикација, сесије, сервиси и тајне 22
5.1 Приступ има рок и контекст 22
5.2 Примена принципа и честе грешке 22
5.3 Пример: сесија после промене привилегије 23
5.4 Дизајнерска одлука: поверење се обнавља 23
5.5 Тајне као део архитектуре 23
5.9 Програмски пример и самостални задатак 25
6 Детекција, инцидент и рањивости 27
6.1 Сигнал тражи контекст 27
6.2 Примена принципа и честе грешке 28
6.3 Пример: више неуспелих пријава 28
6.4 Дизајнерска одлука: сигнал није пресуда 28
6.5 Учење након догађаја 28
6.9 Програмски пример и самостални задатак 30
7 Атрибути, ризик и периодични преглед приступа 32
7.1 Право приступа није заувек 32
7.2 Примена принципа и честе грешке 32
7.3 Пример: иста улога, различит тренутак 33

7.4 Дизајнерска одлука: контекст без хаоса 33
7.5 Ризик као разговор о одлуци 33
7.9 Програмски пример и самостални задатак 35
8 Корелација догађаја, ефективност контрола и учење 37
8.1 Контрола има смисао тек уз доказ 37
8.2 Примена принципа и честе грешке 38
8.3 Пример: контрола постоји, али нема свеж доказ 38
8.4 Дизајнерска одлука: корелација пре закључка 38
8.5 Ефективност је више од постојања контроле 38
8.9 Програмски пример и самостални задатак 40
Сажетак: безбедност као следљив, доказив процес 42
Препоручена литература и документација 42

Како користити практикум

Практикум обрађује основе информационе безбедности кроз осам тематских целина. Свако поглавље полази од безбедносног проблема, објашњава модел и одлуке, приказује дијаграме и тестове и завршава се задатком за самостални рад. Циљ није механичко преписивање примера, већ разумевање онога што систем дозвољава, одбија и доказиво бележи.

Самостална примена

Принципи су представљени општим примерима из различитих информационих система. Сваки пример прати проверљив захтев, границу поверења, одлуку о приступу или безбедносној контроли и доказ у облику негативног теста, ревизијског записа или поновљивог сценарија.

Фаза	Препоручени начин рада
Пре вежбе	Прочитати тему и издвојити претпоставке које би у информационом систему могле бити погрешне.
Током вежбе	Повезати појам са последицама у развоју информационог система, а не само са називом класе или крајње тачке API-ја.
После вежбе	Применити принцип на примеру система и документовати резултат тестом, одлуком или ревизијским записом.
Пред одбрану	Објаснити проблем, изабрану контролу, њено ограничење и начин на који је понашање проверено.

Табела 0.1.1. Фаза и Препоручени начин рада

Дефанзивна оријентација

Практикум не захтева развој експлоита, малвера ни напад на реалне системе. Сумњива активност, грешка конфигурације или покушај недозвољеног приступа репродукују се контролисаним симулаторима и тестним идентитетима.

Језик и алати

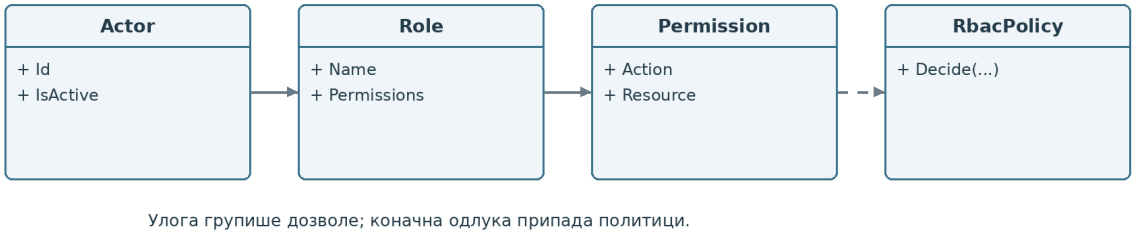
Редослед осам тема прати поступан прелаз од идентитета и ауторизације, преко безбедносне конфигурације, модела претњи и управљања сесијама, до детекције, ризика и мерења ефективности контрола.

Примери користе C# и .NET 8. Пре покретања проверити SDK и структуру одговарајућег наставног примера.

1 Идентитет, аутентикација и контрола приступа заснована на улогама

Безбедност почиње раздвајањем идентитета, пријаве и овлашћења. Систем најпре треба да поуздано препозна субјекат, затим да процени његов актуелни статус, а тек потом да провери шта тај субјекат сме да уради. Када се ти кораци помешају, грешке се лако сакрију иза једне успешне пријаве.

Контрола приступа заснована на улогама важна је зато што права не додељује произвољно сваком налогу. Улоге изражавају одговорности у организацији, па тим може разумљиво да управља приступом, да га прегледа и да га укине када више није потребан.



Слика 1.1.1. UML дијаграм класа: субјекат, улога, дозвола и политика приступа

Пријављивање није исто што и овлашћење: сервер мора да провери идентитет, актуелни статус налога, радњу и опсег дозволе.

Циљ вежбе

Студент треба да образложи одлуку система, податке на којима се она заснива и њен утицај на заштиту података и услуга.

1.1 Ментални модел приступа

Изражавање приступа кроз улоге смањује број појединачних додела права, али само када су дозволе прецизно именоване и ограничене на потребне ресурсе. У сваком захтеву се проверава да ли је актер пријављен и активан, да ли има одговарајућу дозволу и да ли се она односи на тражену радњу.

- Успешна пријава потврђује идентитет, али сама по себи не даје право на ресурс.
- Улога треба да буде читљива пословна одговорност, а дозвола мала и проверљива радња.
- Промена статуса или улоге мења приступ одмах, без ослањања на застарелу претпоставку клијента.

Питање за дискусију

За једну административну радњу објасните разлику између идентитета, улоге, дозволе и опсега у коме дозвола важи.

1.2 Примена принципа и честе грешке

Дизајн почиње питањима ко су актери, које одговорности имају и над којим опсегом раде. Сервер доноси коначну одлуку; интерфејс само помаже кориснику да не бира радње које вероватно неће проћи.

Честе грешке које треба препознати

- Мешање аутентикације и ауторизације у једну проверу.
- Давање широке улоге зато што је брже од обликовања једне дозволе.
- Ослањање на сакривено дугме као на безбедносну контролу.

Повезаност са другим целинама

Разматрана одлука повезује идентитет, податке, политике и доказе, а исти поступак анализе примењује се на различите врсте информационих система.

1.3 Пример: оператор добија нову одговорност

Оператор је до јуче прегледао извештаје, а данас привремено помаже тиму који одобрава промене на осетљивим ресурсима. Најједноставније решење делује као додела администраторске улоге. Међутим, та одлука често отвара много више радњи него што је стварно потребно, а касније је тешко утврдити које право је било оправдано и када треба да нестане.

- Прво се одређује конкретна радња коју оператор треба да изврши, а не назив најшире постојеће улоге.
- Затим се проверава опсег: да ли право важи за цео систем, један домен или само ограничено време.
- На крају се дефинише ревизија или истек, како привремена потреба не би остала трајна привилегија.

Критеријуми решења

За сценарио привременог проширења права потребно је приказати дозволу са ограниченим опсегом, њено повлачење и негативан тест.

1.4 Дизајнерска одлука: улога није дозвола

Улога је начин да организација групише одговорности, док дозвола изражава најмању радњу коју систем штити. То раздвајање делује као додатни посао на почетку, али смањује цену сваке касније промене. Када се дода нова радња, тим не мора да измишља нову комбинацију корисника и изузетака, већ процењује којој одговорности припада и над којим опсегом важи.

- Улоге треба именовати по одговорности, не по техничком слоју апликације.
- Дозвола треба да описује радњу и ресурс довољно прецизно да може да се провери.

- Ефективна права настају из улоге, опсега и актуелног статуса, а не из једног статичког поља корисника.

1.5 Како знамо да модел приступа ради

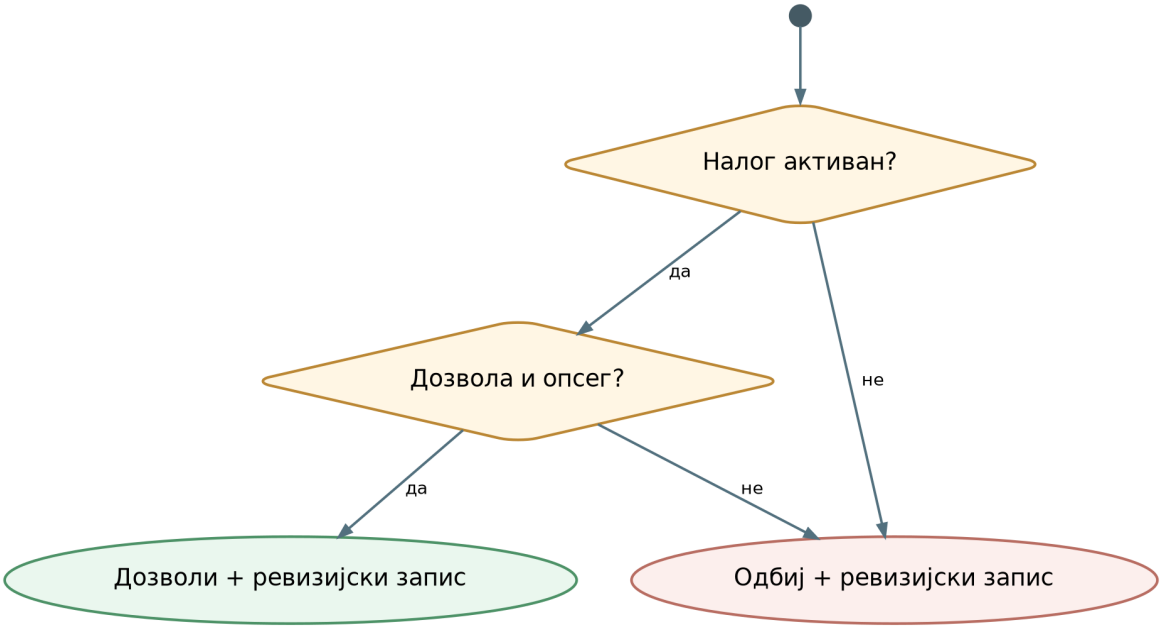
Најкориснији доказ није само успешна пријава. Потребно је показати и да систем одбија радњу када изостане једна потребна чињеница: активан идентитет, одговарајућа дозвола или исправан опсег. Такви негативни случајеви штите модел од случајног ширења права кроз касније измене.

- Тестови пореде очекивано допуштену и одбијену радњу над истим типом ресурса.
- Ревизијски запис бележи ко је покушао радњу, шта је захтевано и какав је исход.
- Преглед права треба да објасни основ приступа, а не само назив корисничке улоге.

Самостална провера

За изабрани сценарио наведите одлуку система, коришћене податке, очекивани исход и доказ на основу ког други члан тима може да потврди резултат.

1.6 Модел и дијаграм одлуке



Слика 1.6.1. UML дијаграм активности: одлука заснована на улози и опсегу

Пријава је само једна фаза. Право приступа настаје тек када систем провери цео контекст.

За сваку одлуку приказати која информација потиче из идентитета, а која из политика приступа.

1.7 Поређење основних појмова идентитета

Појам	Питање на које одговара	Типична грешка
Идентитет	Ко је субјекат?	Мешање особе и техничког налога.

Појам	Питање на које одговара	Типична грешка
Аутентикација	Да ли је тврдња о идентитету потврђена?	Успешна пријава се тумачи као широко овлашћење.
Ауторизација	Да ли је радња допуштена?	Провера се ради само у корисничком интерфејсу.
Сесија	У ком привременом контексту приступ важи?	Токен се третира као трајно право.

Табела 1.7.1. Појам и питање на које одговара

Тумачење и примена

Раздвајање идентитета, улоге, дозволе и опсега спречава да једна широка улога случајно отвори више ресурса.

1.8 Анализа захтева и безбедносни доказ

Када систем прими захтев, корисно је посматрати одлуку као низ питања, а не као једну методу која враћа да или не. Свако питање смањује скуп могућих грешака и оставља јасан траг за каснији преглед.

1. Да ли је субјекат познат и активан?
2. Да ли је захтевана радња дефинисана као дозвола?
3. Да ли улога и опсег дају то право баш у овом тренутку?
4. Који ревизијски запис описује успех или одбијање?

Исход учења

По завршетку теме студент разликује аутентикацију и ауторизацију, моделује улоге и дозволе и доказује правило приступа негативним тестом.

Самостална примена

Применити правило из теме на једној изабраној функционалности: описати субјекат, ресурс, претпоставке и очекивану одлуку, затим припремити негативан тест и запис који објашњава исход.

Проверљив пример: Привремени приступ извештајима

Ситуација. Запослени прелази у тим који припрема месечни извештај. Новој улози је потребна дозвола reports:view, али није потребно мењати глобалну администраторску улогу. Систем посебно чува идентитет, статус налога, припадност улогама и дозволе повезане са тим улогама. Зато је могуће укинути привремени приступ без измене других права тог запосленог.

Поступак провере. Поставити три упита над истим ресурсом: активан корисник са траженом дозволом треба да буде прихваћен; активан корисник без дозволе мора бити одбијен; деактивиран корисник мора бити одбијен чак и ако и даље има одговарајућу улогу. За сваки исход забележити стабилан разлог одлуке, без откривања осетљивих података у одговору.

Зашто је контрапример погрешан. Контрапример је решење које проверава само назив улоге или скривено дугме у интерфејсу. То није серверска провера овлашћења и не штити директан позив API-ја.

1.9 Програмски пример и самостални задатак

Ауторизација приступа извештајима на основу улога. RbacPolicy подразумевано одбија приступ; пријављен налог мора имати тражену дозволу.

Следећи тест издваја један проверљив безбедносни услов. Његов резултат има смисла тек када се објасне улаз, правило и очекивана одлука.

```
[Test]
public void Decide_WhenActorHasNoRoles_DeniesByDefault()
{
    var decision = _policy.Decide(Authenticated(), "reports:view", []);

    Assert.That(decision.Allowed, Is.False);
}
```

Листинг 1.9.1. Одбијање подразумевано, када актер нема улоге. Извор: RbacPolicyTests.cs.

```
cd examples/oib-dotnet/vezba-01-identitet-rbac
dotnet run --project src/Oib.Vezba01.ConsoleUi
dotnet test Oib.Vezba01.sln
```

Листинг 1.9.2. Покретање из репозиторијума наставног портала; потребан је .NET 8 SDK.

Провера	Садржај
Сценарио	Улога без дозволе reports:export покушава извоз.
Тестови	RbacPolicyTests, AuthorizeAccessHandlerTests
Доказ	AccessAuditEntry бележи и одобрен и одбијен приступ.

Табела 1.9.1. Провера и садржај

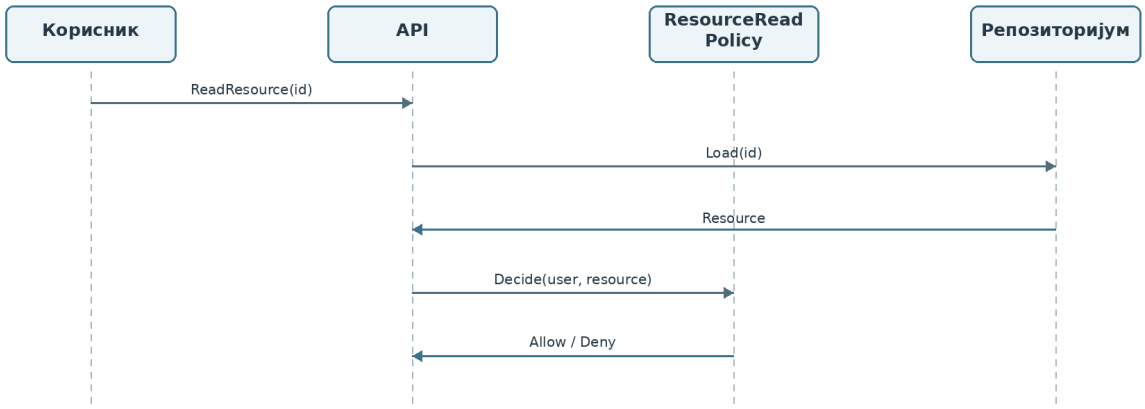
Самостални задатак

Додати атрибут `IsActive` актеру и правило да деактивиран налог нема приступ иако има улогу. Правило додати у `RbacPolicy`, увести нов код одлуке и покрити га тестом.

2 Ауторизација над ресурсом и класификација података

Ауторизација има смисла тек када је везана за конкретан ресурс и његов контекст. Корисник може имати легитимну улогу, а ипак не сме да чита, мења или дели баш тај запис. Зато се одлука не завршава провером типа ендпоинта или назива улоге.

Класификација података даје исти језик власницима, развоју и операцијама. Када систем зна колико је податак осетљив, може да примени одговарајући приступ, приказ, чување и евиденцију без ослањања на нечије сећање.



Слика 2.1.1. UML дијаграм секвенце: ауторизација над конкретним ресурсом

Чак и ако је улога исправна, над конкретним записом остаје да се провере власништво, класификација и тражена радња.

Циљ вежбе

Студент треба да образложи одлуку система, податке на којима се она заснива и њен утицај на заштиту података и услуга.

2.1 Одлука мора имати контекст

Политика над конкретним ресурсом проверава да ли тражену радњу дозвољавају не само улога већ и власништво над записом, класификација и контекст захтева. Одбијање треба да врати стабилан исход без откривања садржаја заштићеног записа.

- Провера права укључује субјект, радњу, ресурс и његову припадност.
- Одбијање не треба да открива садржај нити постојање туђег осетљивог ресурса.
- Класификација није етикета за документацију, већ улаз у правила руковања подацима.

Питање за дискусију

Који ресурс носи ризик хоризонталног приступа и које информације одлука мора да узме у обзир?

2.2 Примена принципа и честе грешке

За сваку значајну операцију тим формулише и позитиван и негативан сценарио. Негативан сценарио проверава да правилно пријављен корисник не може да заобиђе власништво променом идентификатора или позивом API-ја ван интерфејса.

Честе грешке које треба препознати

- Провера улоге без провере односа према конкретном ресурсу.
- Филтрирање листе у клијенту без исте провере на серверу.
- Снижавање класификације без образложења и трага промене.

Повезаност са другим целинама

Разматрана одлука повезује идентитет, податке, политике и доказе, а исти поступак анализе примењује се на различите врсте информационих система.

2.3 Пример: исправна улога, погрешан запис

Два члана тима имају исту улогу и обојица могу да прегледају захтеве. Један мења идентификатор у адреси захтева и долази до туђег записа. Систем који проверава само улогу сматра захтев дозвољеним, иако се променио најважнији део контекста: однос корисника према конкретном ресурсу.

- Одлука укључује идентитет, тражену радњу, ресурс и припадност ресурса одговарајућем домену или власнику.
- Исти принцип важи за API позив, извоз података и позадинску обраду, не само за екран који корисник види.
- Одговор на одбијени приступ треба да буде користан клијенту, али не сме да открије податке о заштићеном запису.

Критеријуми решења

За два различита корисника и исти запис доказати разлику у одлуци; одбијање не сме открити поверљиве податке.

2.4 Дизајнерска одлука: податак носи обавезе

Класификација не говори само колико је податак важан. Она утиче на начин приказа, извоз, дељење, чување и брисање. Када те последице нису јасно везане за класификацију, сваки нови екран или интеграција мора поново да погађа како треба руковати информацијом.

- Класификација има пословно значење које сви чланови тима тумаче на исти начин.
- Правила за приказ и извоз треба да буду строжа када последице грешке расту.
- Власник података одговара за оправданост промене класификације, а систем чува траг те промене.

2.5 Како знамо да је ауторизација потпуна

Приступ треба проверити кроз стварну границу система, а не само кроз кориснички интерфејс. То значи да директан API захтев, позадински процес и извоз не могу

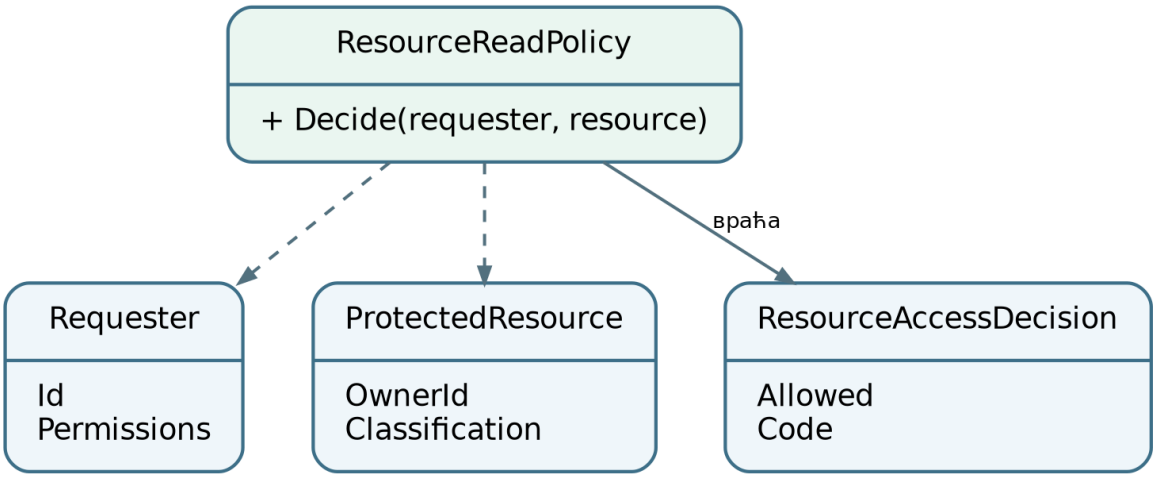
прескочити исто правило. Негативни тестови зато нису додатак после имплементације, већ најјаснији опис границе приступа.

- Систем једнако штити ресурс када се захтев пошаље ван стандардног екрана.
- Одбијени захтев не враћа детаље који помажу да се мапирају туђи подаци.
- Промена класификације и промена власништва остављају довољно трага за каснији преглед.

Самостална провера

За изабрани сценарио наведите одлуку система, коришћене податке, очекивани исход и доказ на основу ког други члан тима може да потврди резултат.

2.6 Модел и дијаграм одлуке



Слика 2.6.1. UML дијаграм класа: контекст захтева и резултат одлуке

Идентификатор у адреси не представља право приступа. Однос субјекта према ресурсу проверава се на серверу.

Означити који кораци припадају API слоју, а који доменској политици приступа.

2.7 Нивои провере приступа

Ниво	Шта проверава	Зашто није довољан сам
Пријава	Идентитет субјекта.	Не говори шта субјекат сме да уради.
Улога	Општи скуп одговорности.	Не зна однос према конкретном запису.
Ресурс	Власништво, припадност и класификацију.	Мора се комбиновати са радњом и политиком.
Контекст	Време, стање и пословно оправдање.	Може променити закључак за исту улогу.

Табела 2.7.1. Ниво и Шта проверава

Тумачење и примена

Ниво класификације и власништво нису заменљиви: први описује обавезе заштите, други право над конкретним записом.

2.8 Анализа захтева и безбедносни доказ

Заштита података не завршава се у тренутку читања. Исти податак пролази кроз креирање, употребу, дељење, чување и уклањање. Свака фаза поставља друго питање о дозвољеном приступу и потребном трагу.

5. При креирању се одређују власник и класификација.
6. При коришћењу се проверава сврха и опсег приступа.
7. При дељењу се проверава граница поверења и минимални скуп података.
8. При чувању и уклањању примењују се рокови чувања и правила ревизијског евидентирања.

Исход учења

Студент уме да дефинише ауторизацију над конкретним ресурсом и провери неовлашћени приступ и ненамерно откривање података.

Самостална примена

Применити правило из теме на једној изабраној функционалности: описати субјекат, ресурс, претпоставке и очекивану одлуку, затим припремити негативан тест и запис који објашњава исход.

Проверљив пример: Приступ туђем запису

Ситуација. Два корисника имају исту улогу оператера и оба смеју да читају сопствене записе. Један од њих мења идентификатор записа у захтеву и покушава да прочита запис другог корисника. Исправно правило не одговара на питање „да ли је оператер“, већ на питање „да ли овај актер сме ову радњу над овим конкретним записом“.

Поступак провере. Проверити сопствени и туђи ресурс са истим налогом. Успешно читање сопственог записа и одбијање приступа туђем запису морају бити аутоматизовани тестови. Садржај који припада другом власнику не сме се појавити ни у поруци о грешци, ни у ревизијском запису доступном позиваоцу.

Зашто је контрапример погрешан. Контрапример је филтрирање приказа на клијенту без провере власништва на серверу. Такав дизајн омогућава да се изменом параметра захтева заобиђе кориснички интерфејс.

2.9 Програмски пример и самостални задатак

`ResourceReadPolicy` проверава дозволу, власништво и класификацију конкретног записа. Провера се извршава на серверу за сваки захтев.

Следећи тест издваја један проверљив безбедносни услов. Његов резултат има смисла тек када се објасне улаз, правило и очекивана одлука.

```
[Test]
public void Decide_WhenRequesterIsNotOwner_Denies()
{
    var decision = _policy.Decide(
        Requester("marko", Permissions.ReadRecords),
        Resource(DataClassification.Internal));

    Assert.That(decision, Is.EqualTo(new ResourceAccessDecision(false,
ResourceAccessCodes.NotOwner)));
}
```

Листинг 2.9.1. Провера власништва над записом. Извор: ResourceReadPolicyTests.cs.

```
cd examples/oib-dotnet/vezba-02-resurs-klasifikacija
dotnet run --project src/Oib.Vezba02.ConsoleUi
dotnet test Oib.Vezba02.sln
```

Листинг 2.9.2. Покретање из репозиторијума наставног портала; потребан је .NET 8 SDK.

Провера	Садржај
Сценарио	Корисник погађа идентификатор туђег записа.
Тестови	ResourceReadPolicyTests, ReadResourceHandlerTests
Доказ	ResourceAccessAuditEntry садржи исход и стабилан код одлуке.

Табела 2.9.1. Провера и садржај

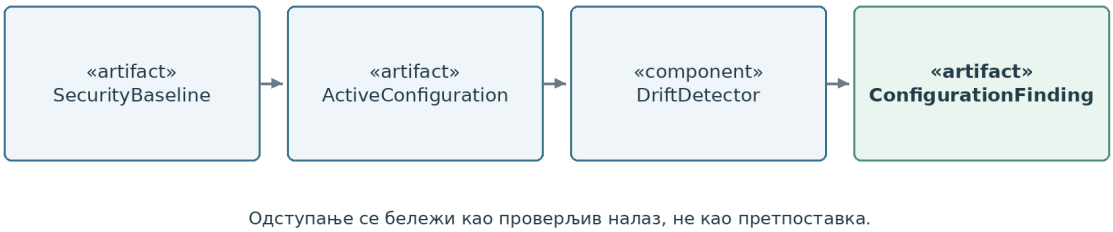
Самостални задатак

Увести радњу измене записа: мења га само власник, без обзира на records:read:any. Додати нову политику и случај употребе поред постојећих, без мењања `ResourceReadPolicy`.

3 Политике, конфигурација и безбедносна видљивост

Безбедносна правила постају поуздана тек када су видљива, верзионисана и проверљива. Правило расуто кроз неколико метода није лако објаснити, прегледати нити променити без нежељених последица. Политика зато представља договорено правило, а конфигурација његову примену у конкретном окружењу.

Видљивост не служи само реаговању на инцидент. Она омогућава тиму да разуме одлуку система, прати одступање од очекиваног стања и повеже догађаје кроз различите слојеве апликације.



Слика 3.1.1. UML дијаграм компоненти: од конфигурације до проверљивог налаза

Одобрена основна конфигурација представља верзионисан уговор; одступање захтева упоредив налаз, власника одговорности и доказ.

Циљ вежбе

Студент треба да образложи одлуку система, податке на којима се она заснива и њен утицај на заштиту података и услуга.

3.1 Правило без доказа је само намера

Полазна безбедносна конфигурација представља мерило са којим се упоређује активно стање. Контрола има једну проверљиву одговорност: да између два стања препозна конкретно одступање и произведе разумљив налаз.

- Нова верзија политике не брише разлог због ког је стара одлука некада била исправна.
- Конфигурациона вредност има власника, значење и очекивани опсег.
- Корелациони идентификатор повезује један безбедносни ток кроз слојеве система.

Питање за дискусију

Које безбедносно правило би требало да буде засебан, читљив артефакт у вашој целини и који доказ показује да ради?

3.2 Примена принципа и честе грешке

Тим бира неколико кључних одлука које мора моћи да објасни из лога и теста. То нису сви догађаји система, већ они чији би нејасан исход отежао истрагу, преглед приступа или корекцију конфигурације.

Честе грешке које треба препознати

- Уређивање објављене политике на месту без историје.
- Тајне у конфигурационим датотекама или логовима.
- Метрика без јасног питања на које треба да одговори.

Повезаност са другим целинама

Разматрана одлука повезује идентитет, податке, политике и доказе, а исти поступак анализе примењује се на различите врсте информационих система.

3.3 Пример: промена правила без губитка историје

Тим пооштрава правило за приступ административном интерфејсу. Ако старо правило само препише новим, касније не може да објасни зашто је ранији приступ био дозвољен нити да реконструише које је очекивање тада важило. Верзионисање не служи бирократији, већ очувању смисла историјских одлука.

- Политика описује намеру и важење, док конфигурација чува вредности којима се та намера примењује у окружењу.
- Свака промена која утиче на одлуку има власника, датум и јасан разлог.
- Лог, метрика или тест дају доказ да систем ради према активном правилу, а не само да је правило записано.

Критеријуми решења

Промена конфигурације треба да произведе детерминистички налаз када нарушава одобрену границу и нулти налаз када је исправна.

3.4 Дизајнерска одлука: правило мора имати границе

Политика постаје корисна када је довољно прецизна да људи доносе доследне одлуке, а довољно читљива да се може објаснити без читања изворног кода. Добро правило наводи на шта се односи, ко је власник, када важи и шта се дешава ако се правила сукобе.

- Конфликт политика решава се унапред договореним правилом, често рестриктивнијом одлуком.
- Конфигурација не мења смисао политике без видљивог процеса одобравања.
- Безбедносна контрола има власника и очекивани доказ, иначе нико не прати да ли је престала да ради.

3.5 Видљивост као способност тима

Опсервабилност подразумева да се за значајну одлуку може пронаћи одговарајући контекст: место настанка, ток кроз слојеве система и разлог успешног или одбијеног исхода. Корелациони идентификатор повезује записе који припадају истој операцији.

- Технички логови разликују се од безбедносно значајних ревизијских записа.
- Метрика има назив, власника и прецизно дефинисано питање на које одговара.
- Тајне, токени и непотребни лични подаци не бележе се у логове нити у ознаке метрика.

Самостална провера

За изабрани сценарио наведите одлуку система, коришћене податке, очекивани исход и доказ на основу ког други члан тима може да потврди резултат.

3.6 Модел и дијаграм одлуке



Слика 3.6.1. UML дијаграм активности: откривање одступања од безбедносне основе

Правило може бити добро написано, а контрола ипак неефективна. Доказ повезује очекивање са реалним понашањем.

Пратити читав пут од конфигурације и baseline-а до налаза и ревизијског трага.

3.7 Политика, процедура и контрола

Елемент	Улога	Пример доказа
Политика	Одређује правило и циљ заштите.	Верзионисан текст са власником.
Процедура	Описује како људи или систем спроводе правило.	Преглед корака или аутоматизовани ток.
Контрола	Спречава, открива или ограничава ризик.	Тест, лог или резултат провере.
Метрика	Показује тренд или одступање.	Дефинисан праг и извор података.

Табела 3.7.1. Елемент и улога

Тумачење и примена

Политика формулише захтев, контрола га проверава, а ревизијски доказ показује шта је стварно измерено.

3.8 Анализа захтева и безбедносни доказ

Метрика је корисна само ако води ка одлуци. Зато се не почиње бројем, већ питањем које оператер или власник система мора да реши. Затим се бира извор, период посматрања и праг при коме је потребна акција.

9. Које понашање желимо да разумемо или ограничимо?
10. Који догађаји поуздано представљају то понашање?
11. Ко је власник прага и шта ради када се праг пређе?
12. Како разликујемо нормалну промену рада од безбедносног одступања?

Исход учења

Студент уме да упореди активну и одобрену конфигурацију и пројектује проверљиву контролу одступања.

Самостална примена

Применити правило из теме на једној изабраној функционалности: описати субјекат, ресурс, претпоставке и очекивану одлуку, затим припремити негативан тест и запис који објашњава исход.

Проверљив пример: Промена безбедносне конфигурације

Ситуација. Организација пооштрава минималну дужину лозинке и дозвољено трајање сесије. Политика дефинише очекивано стање, док конфигурација представља тренутно стање система. Између њих мора постојати проверљиво поређење и запис о томе које је одступање откривено.

Поступак провере. Припремити три конфигурације: потпуно усклађену, конфигурацију са једним одступањем и конфигурацију којој недостаје обавезна вредност. Контрола треба да пријави само стварне налазе, сачува идентификатор провере и не измишља успех ако су улазни подаци непотпуни.

Зашто је контрапример погрешан. Контрапример је провера којој је довољно да постоји конфигурациона датотека. Постојање датотеке само по себи не доказује да се њени безбедносни параметри примењују.

3.9 Програмски пример и самостални задатак

`ConfigurationDriftDetector` пореди активну конфигурацију са усвојеним полазном конфигурацијом. Контроле имплементирају `IConfigurationControl`; налаз означава слабљење заштите.

Следећи тест издваја један проверљив безбедносни услов. Његов резултат има смисла тек када се објасне улаз, правило и очекивана одлука.

```
[TestCase(14)]
[TestCase(20)]
public void MinimumPasswordLength_WhenAtLeastBaseline_ReportsNothing(int length)
{
    var finding = new MinimumPasswordLengthControl().Check(
        Baseline,
        new SecurityConfiguration(length, true));

    Assert.That(finding, Is.Null);
}
```

Листинг 3.9.1. Исправна конфигурација без налаза. Извор: ConfigurationControlTests.cs.

```
cd examples/oib-dotnet/vezba-03-politike-konfiguracija
dotnet run --project src/Oib.Vezba03.ConsoleUi
dotnet test Oib.Vezba03.sln
```

Листинг 3.9.2. Покретање из репозиторијума наставног портала; потребан је .NET 8 SDK.

Провера	Садржај
Сценарио	Минимална дужина лозинке или захтев за администраторски вишефакторска аутентикација ослаби у односу на баселине.
Тестови	ConfigurationControlTests, DetectConfigurationDriftHandlerTests
Доказ	DriftReport садржи цоррелатион ИД и верзију баселине-а.

Табела 3.9.1. Провера и садржај

Самостални задатак

Додати контролу за максимално трајање сесије: проширити `SecurityBaseline` и `SecurityConfiguration`, написати нову `IConfigurationControl` класу и тест који показује да дужа сесија од дозвољене даје налаз.

4 Имовина, границе поверења и моделовање претњи

Добра безбедносна одлука не полази од листе познатих решења, већ од вредне имовине, тока података и граница поверења. Тек тада тим може да препозна шта би могло поћи по злу и која контрола има смисла за конкретан ризик.

Моделовање претњи повезује техничку структуру система са пословним последицама и омогућава да се погрешне претпоставке уоче пре него што постану део стабилне архитектуре.



Слика 4.1.1. Дијаграм граница поверења: корисник, интерни систем и интеграција

У моделу претњи најпре се идентификују вредна имовина, токови и границе поверења, а тек потом могућа злоупотреба и контрола.

Циљ вежбе

Студент треба да образложи одлуку система, податке на којима се она заснива и њен утицај на заштиту података и услуга.

4.1 Модел пре контроле

Модел претњи није каталог свих могућих напада, већ избор релевантних сценарија на основу вредности имовине и уочених граница поверења. Добра процена раздваја претпоставке, пословни утицај и преостали ризик после увођења контроле.

- Имовина има власника, значај и животни циклус.
- Спољна компонента не добија поверење само зато што је интегрисана.
- Претња постаје корисна када води до захтева, контроле и начина провере.

Питање за дискусију

Нацртајте један ток осетљивог податка и означите место на коме се претпоставка о поверењу мења.

4.2 Примена принципа и честе грешке

Архитектурни дијаграм постаје основа безбедносне анализе када приказује податке, актере и претпоставке уз сваку важну границу. Тим затим бира неколико реалних сценарија злоупотребе и проверава их тестом или прегледом дизајна.

Честе грешке које треба препознати

- Инвентар који постоји само ради евиденције, без власништва и значаја.
- Општи каталог претњи који није повезан са стварним системом.
- Контрола изабрана пре разумевања ризика који треба да смањи.

Повезаност са другим целинама

Разматрана одлука повезује идентитет, податке, политике и доказе, а исти поступак анализе примењује се на различите врсте информационих система.

4.3 Пример: извоз из интерног система

Интерни сервис припрема извоз осетљивих података за други систем. Тим га доживљава као безбедан јер је сервис „унутра“, али сам прелазак података у други систем мења претпоставке. Потребно је знати који податак путује, ко прима податак, шта може поћи по злу и како систем спречава или бар открива нежељен исход.

- Имовина обухвата налоге, кључеве, сервисне идентитете, конфигурацију и пословне записе, а не само базу података.
- Граница поверења може бити API позив, ред порука, административни интерфејс или извоз у датотеку.
- Модел претњи обухвата одабране реалне сценарије, одговарајуће контроле и начин њихове провере.

Критеријуми решења

За изабрану претњу навести предуслов, утицај, постојећу контролу и тест који потврђује њено деловање.

4.4 Дизајнерска одлука: прво модел, затим алат

Избор безбедносног алата заснива се на имовини коју треба заштитити, граници коју треба надзирати и сценарију који треба открити. Модел претњи усмерава избор контроле и омогућава процену њене вредности.

- Критичност имовине одређује колико брзо се реагује и колико је доказа потребно.
- Ток података открива места на којима информација може бити измењена, изгубљена или погрешно изложена.
- Претња добија приоритет према утицају, вероватноћи и реалним постојећим контролама.

4.5 Одржавање модела претњи

Модел претњи није документ који се напише једном на почетку семестра. Мења се када се дода нови улаз, интеграција, начин чувања података или привилегована радња. Свака већа архитектонска промена је прилика да се провери да ли су старе претпоставке и даље тачне.

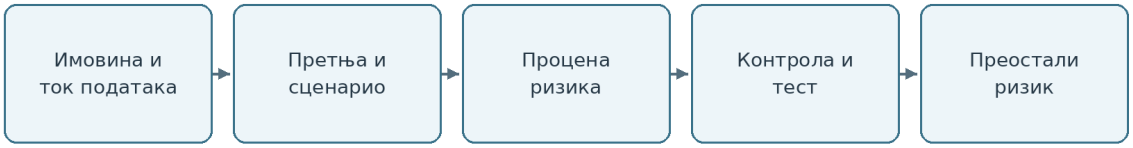
- Дијаграм се ажурира када ток података или граница поверења стварно промени понашање система.
- За одабрани сценарио постоји веза до контроле, теста или прегледа који га проверава.

- Преостали ризик се записује јасно, уместо да се претвара да ниједна претња више не постоји.

Самостална провера

За изабрани сценарио наведите одлуку система, коришћене податке, очекивани исход и доказ на основу ког други члан тима може да потврди резултат.

4.6 Модел и дијаграм одлуке



Слика 4.6.1. Дијаграм анализе ризика: имовина, претња, контрола и доказ

Претња добија смисао тек када се веже за конкретну имовину, ток и последицу.

Означити место преласка границе поверења и показати где се проверава неовлашћени улаз.

4.7 Имовина, рањивост и претња

Појам	Значење	Питање за анализу
Имовина	Оно чија вредност захтева заштиту.	Шта губимо ако је угрожена?
Рањивост	Слабост коју је могуће искористити.	Где систем нема довољну заштиту?
Претња	Могући узрок нежељеног исхода.	Ко или шта може активирати слабост?
Ризик	Комбинација вероватноће и последице.	Шта прво треба умањити?

Табела 4.7.1. Појам и значење

Тумачење и примена

Имовина означава вредност, рањивост слабост, а претња могући извор штетног догађаја; ризик настаје у контексту.

4.8 Анализа захтева и безбедносни доказ

Архитектонска скица добија безбедносну вредност када се преко компоненти уцртају подаци, идентитети и границе поверења. Није потребно цртати сваку класу. Довољно је показати делове кроз које пролази вредна информација и одлуке које мењају њен статус.

13. Означити улазе које контролише корисник или спољашњи сервис.
14. Означити место где се идентитет или дозвола поново проверавају.
15. Означити складишта, кључеве и интеграције које мењају последицу инцидента.
16. Уз сваку границу записати једну реалну претпоставку која може бити погрешна.

Исход учења

Студент уме да из архитектонског тока издвоји имовину, претњу и границу поверења и предложи мерљиву контролу.

Самостална примена

Применити правило из теме на једној изабраној функционалности: описати субјекат, ресурс, претпоставке и очекивану одлуку, затим припремити негативан тест и запис који објашњава исход.

Проверљив пример: Извоз података преко границе поверења

Ситуација. Унутрашња апликација формира извештај и предаје га спољном сервису. Критична граница није само мрежна веза: важно је ко покреће извоз, који су подаци обухваћени, где се чувају, како се проверава одредиште и како се бележи догађај. Модел претњи зато прати податак од извора до пријемника.

Поступак провере. Нацртати дијаграм тока података са актером, системом, складиштем и спољним сервисом. За сваку границу поверења навести претњу и конкретну контролу. У тесту проверити да захтев за извоз без дозволе није извршен и да се садржај не налази у недозвољеном излазу.

Зашто је контрапример погрешан. Контрапример је листа општих опасности без повезивања са током података. Без модела имовине и граница не може се уверљиво објаснити због чега је одређена контрола потребна.

4.9 Програмски пример и самостални задатак

> Процена сценарија претње повезује имовину, ток података и границу поверења. `ThreatRiskCalculator` рачуна ризик; демонстрациона политика захтева третман при вредности 24 или већој.

Следећи тест издваја један проверљив безбедносни услов. Његов резултат има смисла тек када се објасне улаз, правило и очекивана одлука.

```
[TestCase(0)]
[TestCase(6)]
public void Validate_WhenLikelihoodIsOffTheScale_ReturnsStableCode(int likelihood)
{
    var result = _validator.Validate(Scenarios.Create(likelihood, 3, false));

    Assert.That(result.Error, Is.EqualTo(ThreatModelErrorCodes.LikelihoodOutOfRange));
}
```

Листинг 4.9.1. Валидација границе скале ризика. Извор: ThreatScenarioValidatorTests.cs.

```
cd examples/oib-dotnet/vezba-04-threat-modeling
dotnet run --project src/Oib.Vezba04.ConsoleUi
dotnet test Oib.Vezba04.sln
```

Листинг 4.9.2. Покретање из репозиторијума наставног портала; потребан је .NET 8 SDK.

Провера	Садржај
Сценарио	Извоз личних података стиже неовлашћеном примаоцу.
Тестови	ThreatRiskCalculatorTests, AssessThreatModelHandlerTests
Доказ	ThreatModelReport приказује сценарије по ризику и предложене контроле.

Табела 4.9.1. Провера и садржај

Самостални задатак

Додати резидуални ризик: `ThreatScenario` добија процену ефикасности контроле, а извештај приказује ризик пре и после контроле. Правило сместити у доменски сервис и покрити граничним тестовима.

Контролна листа за анализу граница поверења

Пре избора контроле проверити да модел претњи обухвата:

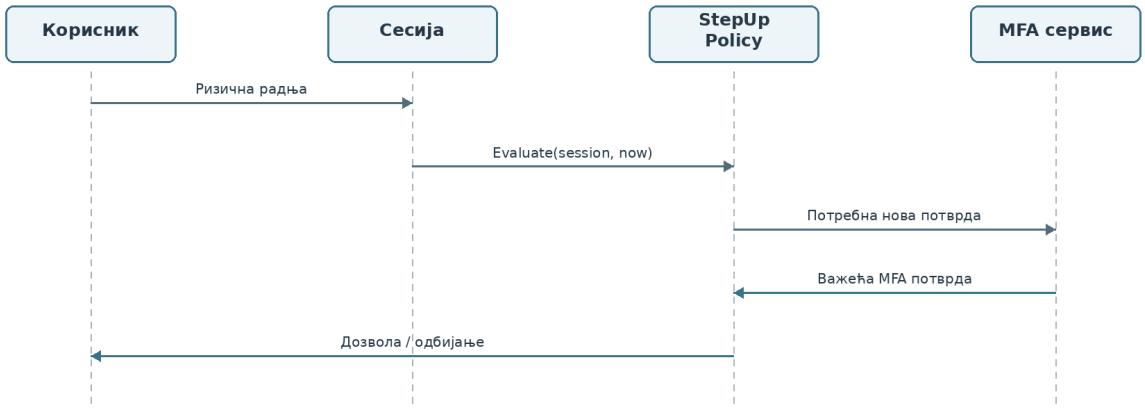
- имовину и пословну последицу њеног нарушавања, а не само техничку компоненту;
- актере, спољне системе и изворе података који прелазе дефинисану границу поверења;
- начин на који претња може да се оствари у посматраном току података;
- контролу која мења вероватноћу или последицу конкретног сценарија;
- негативан тест и траг који потврђује да је контрола заиста примењена.

Коначан резултат овакве анализе је следљив запис: имовина → граница поверења → сценарио претње → контрола → доказ. Тај поступак може се применити и на мале апликације, без посебног алата за моделовање претњи.

5 Вишефакторска аутентикација, сесије, сервиси и тајне

Идентитет није трајно доказан једном пријавом. Ризик радње, трајање сесије, врста уређаја и промена привилегија утичу на то колико је додатна потврда потребна. Оперативна безбедност зато прати приступ и након почетне аутентикације.

Вишефакторска потврда и временски ограничене привилегије смањују последице украденог креденцијала. Управљање тајнама штити везе међу сервисима, где непажња често остане скривена јер нема корисничког екрана.



Слика 5.1.1. UML дијаграм секвенце: временски ограничена MFA потврда

Успешна пријава није трајна дозвола за сваку радњу: ризични захтев може захтевати нову, временски свежу MFA потврду.

Циљ вежбе

Студент треба да образложи одлуку система, податке на којима се она заснива и њен утицај на заштиту података и услуга.

5.1 Приступ има рок и контекст

Сесија повезује тренутни контекст корисника са доказима о пријави и додатним факторима. Време од последње MFA потврде, важност сесије и ризик операције одређују да ли је потребно поновно потврђивање идентитета.

- Сесија може изгубити важност пре планираног истека.
- Додатна потврда прати ризик радње, не само тип корисника.
- Сервисни идентитет има мали, јасно оправдан опсег, као и људски идентитет.

Питање за дискусију

Која радња заслужује додатну потврду идентитета и који догађај би требало да опозове постојећу сесију?

5.2 Примена принципа и честе грешке

Развојни тим унапред одређује које ситуације мењају ниво поверења: промена привилегије, приступ осетљивом податку или сумњива пријава. Тајне се референцирају

из безбедног извора и никада се не преносе кроз код, тестне исписе или историју репозиторијума.

Честе грешке које треба препознати

- Токен третиран као вечна дозвола.
- Администраторска права додељена без рока и прегледа.
- Једна широка тајна дељена међу више независних сервиса.

Повезаност са другим целинама

Разматрана одлука повезује идентитет, податке, политике и доказе, а исти поступак анализе примењује се на различите врсте информационих система.

5.3 Пример: сесија после промене привилегије

Корисник се пријавио ујутру, а касније током дана добија виши ниво приступа. Ако систем не преиспита постојећу сесију, корисник може извршити ризичну радњу без додатне потврде која би била потребна да се пријављује у том тренутку. Обрнуто, одузета привилегија мора заиста да престане да важи, а не да чека крај дуготрајне сесије.

- Сесија представља привремени контекст приступа, а не трајну потврду поверења.
- Додатна аутентикација примењује се када ризик радње захтева виши ниво потврде идентитета.
- Сервисни налог има најмањи потребан опсег, рок важења и контролисану ротацију тајне.

Критеријуми решења

Тестирати захтев са свежом MFA потврдом, застарелом потврдом и неактивном сесијом.

5.4 Дизајнерска одлука: поверење се обнавља

Сесија олакшава рад кориснику, али њен комфор не сме постати трајно поверење. Систем мора знати када се околности мењају довољно да претходна потврда више није довољна. То могу бити промена привилегије, необичан уређај, ризична радња или сигнал да је налог угрожен.

- Истек ограничава трајање поверења чак и када нема видљивог инцидента.
- Опозив реагује на конкретну промену ризика и не чека да сесија природно нестане.
- Додатни фактор доказује присуство или поседовање независног средства, а не понавља само исту лозинку.

5.5 Тајне као део архитектуре

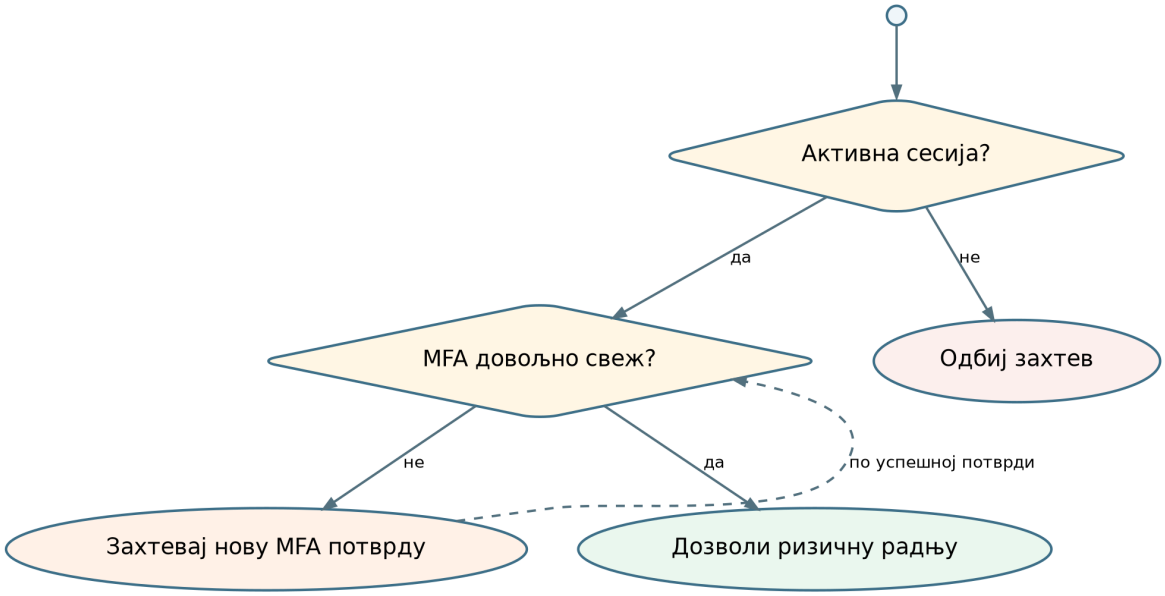
Тајна није конфигурациона вредност као свака друга. Њено излагање може дати приступ без видљивог корисничког екрана, нарочито између сервиса. Зато развојни процес дефинише одакле тајна долази, ко је користи, када се ротира и како се проверава да није случајно завршила у репозиторијуму или логу.

- Сваки сервисни идентитет има сврху и најмањи потребан опсег.
- Ротација се планира пре истека, уместо да се чека прекид рада.
- Тестови проверавају понашање при опозиву или истеку без приказивања стварних вредности тајне.

Самостална провера

За изабрани сценарио наведите одлуку система, коришћене податке, очекивани исход и доказ на основу ког други члан тима може да потврди резултат.

5.6 Модел и дијаграм одлуке



Слика 5.6.1. UML дијаграм активности: поновна провера идентитета за ризичну радњу

Поверење се не чува заувек. Систем га обнавља или опозива када се промени ризик.

Показати где се проверава старост потврде и шта се догађа када је прекорачен дозвољени интервал.

5.7 Различите врсте идентитета

Идентитет	Намена	Посебан ризик
Људски корисник	Интерактивне радње и одговорност.	Крађа креденцијала или преширока улога.
Сервисни налог	Комуникација међу компонентама.	Тајна без рока или превелик опсег.
Привремени приступ	Кратка ризична интервенција.	Право остане активно након потребе.
Администраторски налог	Измене са великим утицајем.	Недостатак додатне потврде и аудита.

Табела 5.7.1. Идентитет и Намена

Тумачење и примена

Људски налог, сервисни идентитет и тајни кључ имају различите власнике, рокове важења и обавезе ротације.

5.8 Анализа захтева и безбедносни доказ

Безбедно управљање тајном почиње пре њеног првог коришћења. Потребно је знати власника, сврху, компоненту која је користи и начин ротације. Сама вредност тајне треба да остане ван кода, екрана и записа који нису за то намењени.

17. Тајна се издаје само идентитету који јој је стварно потребан.
18. Систем прати истек и планира ротацију пре прекида рада.
19. Логови бележе употребу и неуспех без приказивања вредности тајне.
20. Опозив ограничава последицу сумње да је тајна компромитована.

Исход учења

Студент уме да спроведе поновну проверу за ризичну радњу и контролише важење сесија и управљање тајнама.

Самостална примена

Применити правило из теме на једној изабраној функционалности: описати субјекат, ресурс, претпоставке и очекивану одлуку, затим припремити негативан тест и запис који објашњава исход.

Проверљив пример: Осетљива операција и старост потврде

Ситуација. Корисник се успешно пријавио, али више сати касније тражи осетљиву измену налога. Постојање сесије није довољан доказ да је ризична операција дозвољена. Политика проверава старост сесије, време последње MFA потврде, категорију радње и евентуалну промену привилегија.

Поступак провере. За исту радњу тестирати свежу и застарелу MFA потврду, истеклу сесију и сесију након промене улоге. Јасно раздвојити одлуку о поновном потврђивању идентитета од одлуке о дозволи за саму радњу.

Зашто је контрапример погрешан. Контрапример је неограничено поверење у већ издату сесију. Чак и јака почетна пријава не укида потребу за управљањем трајањем и опсегом овлашћења.

5.9 Програмски пример и самостални задатак

`StepUpAuthenticationPolicy` проверава да ли сесија има довољно свежу MFA потврду за ризичну операцију. Одлука разликује одбијање и потребу за новом потврдом.

Следећи тест издваја један проверљив безбедносни услов. Његов резултат има смисла тек када се објасне улаз, правило и очекивана одлука.

```
[Test]
public void Evaluate_WhenMfaIsExactlyAtMaximumAge_Allows()
{
    var decision = _policy.Evaluate(Session(Now.AddMinutes(-10)), Export, Now);

    Assert.That(decision.Allowed, Is.True);
}
```

Листинг 5.9.1. Гранична провера свежине MFA потврде. Извор: StepUpAuthenticationPolicyTests.cs.

```
cd examples/oib-dotnet/vezba-05-mfa-sesije-tajne
dotnet run --project src/Oib.Vezba05.ConsoleUi
dotnet test Oib.Vezba05.sln
```

Листинг 5.9.2. Покретање из репозиторијума наставног портала; потребан је .NET 8 SDK.

Провера	Садржај
Сценарио	Преузета, опозвана или давно потврђена сесија покушава поверљив извоз.
Тестови	StepUpAuthenticationPolicyTests, AuthorizeRiskyOperationHandlerTests
Доказ	StepUpDecision враћа код одлуке и ознаку да ли се тражи вишефакторска аутентикација.

Табела 5.9.1. Провера и садржај

Самостални задатак

Додати максималну старост саме сесије: сесија издата пре више од осам сати одбија се без обзира на претходну вишефакторску аутентикацију. Увести нов код одлуке и тест за граничну вредност.

Критеријуми успешне провере сесије и тајни

При провери решења из ове теме није довољно да тест утврди да је корисник пријављен. Потребно је показати зашто је осетљива радња прихваћена или одбијена у конкретном тренутку.

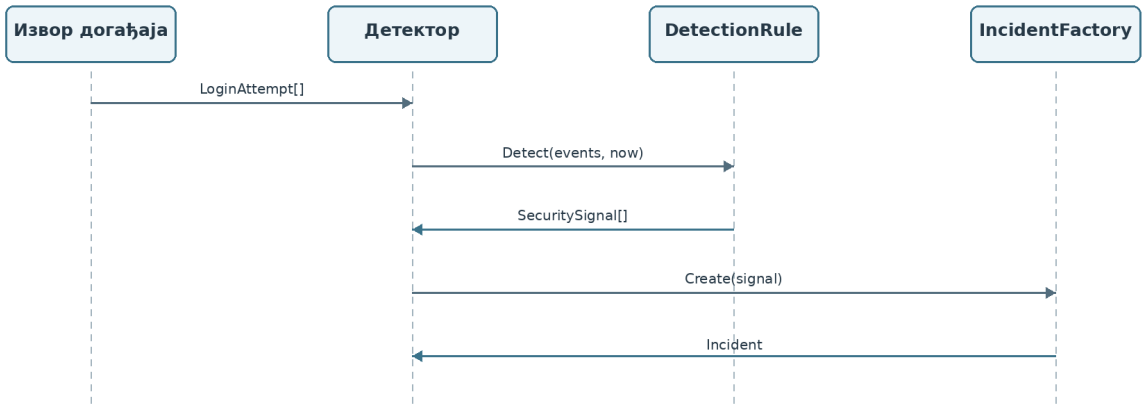
- Проверити важење сесије и време последње додатне потврде за осетљиве радње.
- Утврдити како промена улоге, одјава или опозив сесије утичу на следећи захтев.
- Проверити да сервисни идентификатор нема дозволе које нису неопходне за задату радњу.
- Показати да се тајна не појављује у логу, поруци о грешци или излазном извештају.
- Покренути позитиван и негативан тест над истим случајем употребе и упоредити исходе.

Доказ је поновљив тест са јасним улазом, очекиваном одлуком и разлогом одбијања. Тако се раздваја успешна пријава од стварно важећег овлашћења.

6 Детекција, инцидент и рањивости

Безбедносни догађај није аутоматски инцидент. Систем најпре препознаје сигнал, затим га ставља у контекст и тек онда одређује да ли је потребна реакција. Такав ток спречава и игнорисање проблема и непотребно алармирање због сваког одступања.

Управљање инцидентом даје структуру тренутку када је најлакше изгубити контекст. Рањивост постаје решив проблем када је повезана са имовином, приоритетом, власником и доказом да је корекција проверена.



Слика 6.1.1. UML дијаграм секвенце: од неуспелих пријава до инцидента

Безбедносни сигнал није исто што и потврђен инцидент: између детекције и реаговања постоји анализа контекста.

Циљ вежбе

Студент треба да образложи одлуку система, податке на којима се она заснива и њен утицај на заштиту података и услуга.

6.1 Сигнал тражи контекст

Детекционо правило прима низ догађаја са временским ознакама, примењује јасну границу или образац и враћа сигнале који се могу проверити тестовима. Тек анализа утицаја и контекста претвара сигнал у инцидент који захтева реаговање.

- Праг детекције треба да има смисао и разлог, не само произвољну бројку.
- Затварање упозорења чува образложење, јер је то знање за следећи сличан случај.
- Рањивост није решена променом статуса, већ провером да је ризик заиста смањен.

Питање за дискусију

Који догађај би у вашем систему био користан сигнал, али не и довољан доказ инцидента? Који контекст недостаје?

6.2 Примена принципа и честе грешке

Тим бира мали број сигнала који су разумљиви оператеру. За сваки описује нормално понашање, разлог активирања, одговорну особу и безбедан начин провере. Тако се детекција дизајнира као производна могућност, а не као споредни лог.

Честе грешке које треба препознати

- Упозорење без власника и правила за почетну обраду.
- Аутоматско затварање без доказа о решавању узрока.
- Скенирање зависности без утврђеног начина процене и праћења налаза.

Повезаност са другим целинама

Разматрана одлука повезује идентитет, податке, политике и доказе, а исти поступак анализе примењује се на различите врсте информационих система.

6.3 Пример: више неуспелих пријава

Низ неуспелих пријава може указивати на заборављену лозинку, проблем интеграције или покушај злоупотребе. Сам број неуспеха није довољан за закључак. Почетна обрада укључује порекло покушаја, осетљивост налога и наставак обрасца након упозорења.

- Детекција описује образац који захтева проверу, без аутоматског проглашавања сваке грешке инцидентом.
- Почетна обрада бележи одлуку и разлог затварања упозорења.
- Затварање рањивости захтева доказ да је изложеност уклоњена или формално прихваћена као преостали ризик.

Критеријуми решења

У поновљивом тесту показати праг за сигнал, искључивање старих догађаја и образложење тријаже.

6.4 Дизајнерска одлука: сигнал није пресуда

Детекција покушава да примети промену понашања, не да аутоматски чита намеру корисника. Квалитетан сигнал зато наводи образац, праг и контекст који недостаје за коначну одлуку. Тиме оператер може брзо да разликује нормалан проблем рада система од потенцијалног безбедносног догађаја.

- Праг се бира према нормалном понашању система, а не према случајном броју.
- Упозорење има озбиљност, власника и рок до ког се мора погледати.
- Инцидент се отвара када контекст покаже стварни утицај или вероватну злоупотребу.

6.5 Учење након догађаја

Добар пост-инцидент преглед није потрага за особом која је погрешила. Његов циљ је да пронађе где су недостајали сигнал, заштитна провера, јасна одговорност или припрема за опоравак. Резултат треба да мења систем или процес на начин који се може проверити.

- Хронологија одваја чињенице од претпоставки насталих под притиском.

- Акције након инцидента имају власника и проверљив завршетак.
- Нови сценарио се додаје у тестове или оперативне вежбе, како се исто знање не би изгубило.

Самостална провера

За изабрани сценарио наведите одлуку система, коришћене податке, очекивани исход и доказ на основу ког други члан тима може да потврди резултат.

6.6 Модел и дијаграм одлуке



Неутврђен сигнал се бележи без преурањеног закључка.

Слика 6.6.1. UML дијаграм активности: тријажа и реаговање на безбедносни сигнал

Није сваки сигнал инцидент, али сваки значајан исход оставља знање корисно за следећу одлуку.

Означити извор догађаја, правило детекције, место тријаже и доказ о предузетој радњи.

6.7 Сигнал, упозорење и инцидент

Стање	Шта представља	Потребна реакција
Сигнал	Сиров догађај или одступање.	Сачувати контекст и проверити правило.
Упозорење	Сигнал који је прешао дефинисан праг.	Почетна обрада и процена приоритета.
Инцидент	Потврђен или вероватан безбедносни утицај.	Координисана реакција и следљив ток.
Налаз рањивости	Потенцијална изложеност коју треба проценити.	Власник, рок и доказ корекције.

Табела 6.7.1. Стање и Шта представља

Тумачење и примена

Догађај је забележена чињеница, сигнал детектован образац, а инцидент захтева контекст и одговор.

6.8 Анализа захтева и безбедносни доказ

Налаз постаје користан тек када се повеже са стварном имовином и њеном изложеношћу. Исти технички проблем нема исти приоритет на изолованом развојном рачунару и на јавно доступној услузи која обрађује осетљиве податке.

21. Потврдити да се налаз односи на конкретну компоненту и активну верзију.
22. Проценити утицај у контексту доступности, података и граница поверења.
23. Одредити корекцију, компензациону контролу или образложено прихватање ризика.
24. Затворити налаз тек када доказ потврди да је изложеност смањена.

Исход учења

Студент уме да разликује сигнал и инцидент, реализује детекционо правило и припреми проверљив одговор.

Самостална примена

Применити правило из теме на једној изабраној функционалности: описати субјекат, ресурс, претпоставке и очекивану одлуку, затим припремити негативан тест и запис који објашњава исход.

Проверљив пример: Низ неуспелих пријава

Ситуација. У кратком интервалу стиже више неуспелих пријава са исте адресе. Појединачан догађај није доказ инцидента, али низ повезаних покушаја може указивати на аутоматизовану злоупотребу. Правило детекције користи временски прозор, број покушаја и идентификаторе налога.

Поступак провере. Проверити да изолован неуспех не ствара непотребно упозорење, да праг у дефинисаном временском прозору ствара сигнал и да стари догађаји изван прозора не утичу на одлуку. Потом показати који додатни контекст је потребан да се одлучи о реаговању.

Зашто је контрапример погрешан. Контрапример је одлука да је сваки неуспели покушај пријаве безбедносни инцидент. То ствара шум и смањује употребљивост процеса реаговања.

6.9 Програмски пример и самостални задатак

`RepeatedFailedLoginRule` препознаје најмање три неуспешне пријаве у десет минута. `IncidentFactory` из сигнала формира инцидент са одређеном озбиљношћу.

Следећи тест издваја један проверљив безбедносни услов. Његов резултат има смисла тек када се објасне улаз, правило и очекивана одлука.

```
[Test]
public void Detect_WhenFailuresAreBelowThreshold_RaisesNothing()
{
    var signals = _rule.Detect(Attempts.Failures("ana", "203.0.113.10", 2, Now),
Now).Value!;

    Assert.That(signals, Is.Empty);
}
```

Листинг 6.9.1. Детектор не подиже сигнал испод прага. Извор: RepeatedFailedLoginRuleTests.cs.

```
cd examples/oib-dotnet/vezba-06-detekcija-incident
dotnet run --project src/Oib.Vezba06.ConsoleUi
dotnet test Oib.Vezba06.sln
```

Листинг 6.9.2. Покретање из репозиторијума наставног портала; потребан је .NET 8 SDK.

Провера	Садржај
Сценарио	Узастопни покушаји пријаве са исте адресе.
Тестови	RepeatedFailedLoginRuleTests, IncidentFactoryTests, DetectIncidentsHandlerTests
Доказ	Incident садржи идентификатор, озбиљност, сажетак и време отварања.

Табела 6.9.1. Провера и садржај

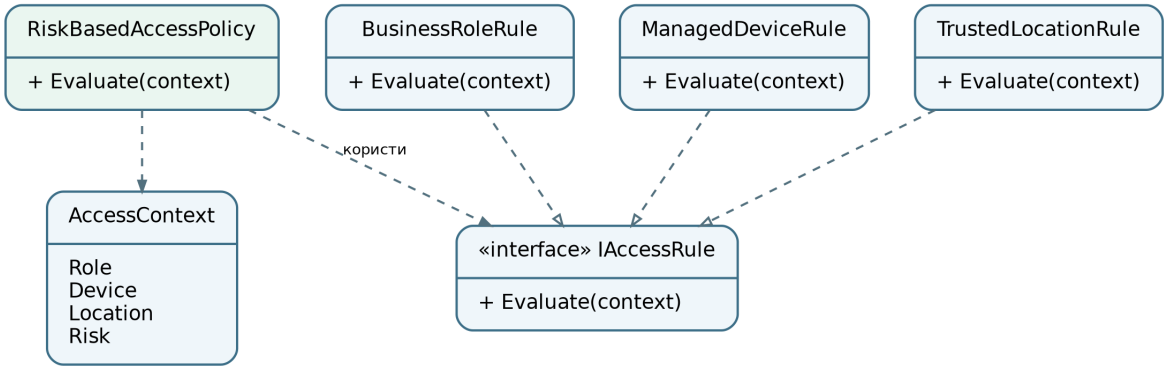
Самостални задатак

Додати правило које препознаје password spraying: иста адреса неуспешно покушава пријаву на више различитих налога. Написати нову `IDetectionRule` класу и регистровати је у корен композиције-у.

7 Атрибути, ризик и периодични преглед приступа

контрола приступа заснована на улогама даје стабилну основу, али сложеније одлуке често зависе и од контекста: припадности, врсте података, времена, стања ресурса или нивоа ризика. Атрибутска правила уводе тај контекст без претварања сваке могуће комбинације у нову улогу.

Регистар ризика и редован преглед приступа спречавају да застареле одлуке остану трајне претпоставке. Евидентирају преостали ризик, одговорно лице и рок за поновно разматрање одлуке.



Слика 7.1.1. UML дијаграм класа: контекстуална правила и политика приступа

Иста улога не значи увек исто право: одлука зависи и од уређаја, локације, осетљивости ресурса и актуелног ризика.

Циљ вежбе

Студент треба да образложи одлуку система, податке на којима се она заснива и њен утицај на заштиту података и услуга.

7.1 Право приступа није заувек

Атрибутска правила допуњују улоге својствима уређаја, локације, ресурса и тренутног ризика. Правила се морају извршавати на серверу, давати предвидљив разлог одбијања и бити довољно независна да се лако проверавају.

- Атрибути се читају из поузданог, јасног контекста.
- Исто стање мора довести до истог резултата политике.
- Ризик укључује имовину, претњу, контролу и преосталу изложеност.

Питање за дискусију

Коју одлуку не бисте смели да заснивате само на улози и који додатни атрибут јој даје смисао?

7.2 Примена принципа и честе грешке

Тим издваја одлуке које не могу поштено да стану у назив једне улоге. Затим их записује као правило које се може тестирати са неколико супротних контекста. Преглед приступа проверава да ли је некадашња пословна потреба и даље стварна.

Честе грешке које треба препознати

- Увођење атрибута без јасног извора и значења.
- Правило које зависи од скривеног стања и даје необјашњив резултат.
- Конфликт дужности откривен тек након што је штета могућа.

Повезаност са другим целинама

Разматрана одлука повезује идентитет, податке, политике и доказе, а исти поступак анализе примењује се на различите врсте информационих система.

7.3 Пример: иста улога, различит тренутак

Два оператера имају исту улогу, али један ради на инциденту који му је додељен, док други ради на другом домену. Ако политика гледа само улогу, обојица би могла да одобре исту ризичну измену. Атрибути као што су припадност домену, стање инцидента и тренутно одобрење дају одлуци контекст који назив улоге сам не може да носи.

- Атрибутска одлука остаје разумљива само ако је извор сваког атрибута познат и поуздан.
- Правило треба да даје исти резултат за исти снимак контекста, како би могло да се тестира и објасни.
- Преглед приступа враћа тим на питање да ли право и даље има пословни разлог, чак и када је некада било правилно додељено.

Критеријуми решења

За исти ресурс проверити приступ из управљаног и неуправљаног уређаја, уз јасне разлоге за одбијање.

7.4 Дизајнерска одлука: контекст без хаоса

Атрибути не треба да постану скривени скуп изузетака који нико не разуме. Сваки атрибут мора имати јасно значење, извор и тренутак на који се односи. Политика затим наводи како се ти подаци комбинују, па одлука може бити поновљена, тестирана и објашњена без ослањања на интуицију имплементатора.

- Атрибути се прикупљају из ауторитативних извора, а не из произвољног захтева клијента.
- Правила треба да буду мала и састављена тако да се конфликт може објаснити.
- Контекст одлуке се бележи довољно да каснији преглед разуме резултат без чувања осетљивих детаља.

7.5 Ризик као разговор о одлуци

Регистар ризика омогућава поређење утицаја, вероватноће, постојећих контрола и преостале изложености. За прихваћени ризик бележе се одговорно лице, образложење и датум наредног прегледа.

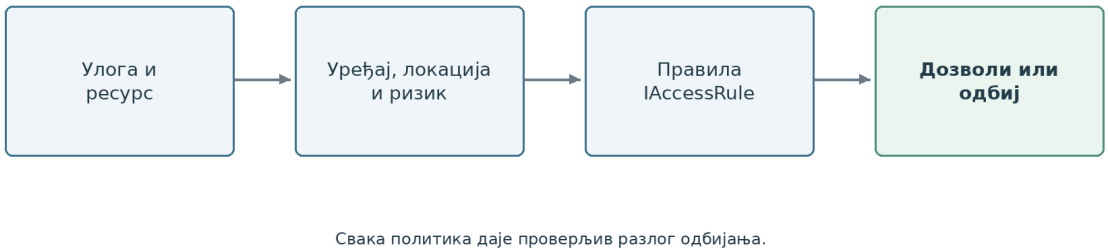
- Ризик повезује конкретну имовину, претњу и контролу.
- Преглед приступа и преглед ризика имају ритам, власника и очекивани исход.

- Сегрегација дужности спречава комбинације права које би једној особи дале превелику контролу.

Самостална провера

За изабрани сценарио наведите одлуку система, коришћене податке, очекивани исход и доказ на основу ког други члан тима може да потврди резултат.

7.6 Модел и дијаграм одлуке



Слика 7.6.1. UML дијаграм активности: заједничка одлука RBAC и ABAC

Контекст даје прецизност одлуци, а редован преглед проверава да ли одлука и даље има смисла.

Показати шта је стабилна улога, а шта променљив атрибут који утиче на одлуку.

7.7 RBAC и ABAC у истој одлуци

Приступ	Снага	Ограничење
контрола приступа заснована на улогама	Јасне, стабилне одговорности.	Тешко изражава време, власништво и стање.
ABAC	Користи богат контекст одлуке.	Тражи поуздане атрибуте и читљива правила.
Комбинација	Улога даје основу, атрибути прецизирају.	Потребно је пажљиво објаснити приоритет правила.
СоД	Спречава ризичне комбинације права.	Мора се проверавати кроз ефективна права.

Табела 7.7.1. Приступ и Снага

Тумачење и примена

RBAC полази од организационе одговорности, а ABAC додаје контекстуалне атрибуте; заједно подржавају принцип најмањих привилегија.

7.8 Анализа захтева и безбедносни доказ

Ризик се не уклања увек у потпуности. Неке мере би биле прескупе, неке би зауставиле важан рад, а неке зависе од спољне стране. Тада је важно да преостали ризик буде видљив, образложен и временски ограничен, а не заборављен.

25. Описати имовину, сценарио и последицу без општих формулација.
26. Навести постојеће контроле и шта оне заиста покривају.
27. Оценити шта остаје незаштићено након тих контрола.
28. Одредити власника који прихвата ризик и датум следећег прегледа.

Исход учења

Студент уме да комбинује улоге и атрибуте, процени контекстуални ризик и објасни периодични преглед права.

Самостална примена

Применити правило из теме на једној изабраној функционалности: описати субјекат, ресурс, претпоставке и очекивану одлуку, затим припремити негативан тест и запис који објашњава исход.

Проверљив пример: Приступ ограничен временом и ризиком

Ситуација. Корисник има улогу аналитичара, али тражени ресурс може да чита само у радно време са управљаног уређаја. Контекст захтева обухвата време, тип уређаја, класификацију ресурса и тренутни ризик. Одлука остаје предвидљива када се ови атрибути јасно прослеђују правилу.

Поступак провере. Извести тестове за радни дан и ван радног времена, управљан и неуправљан уређај, и исту улогу са различитим класификацијама ресурса. Негативни тест треба да показује који је услов изостао, док се у поруци клијенту не откривају непотребни унутрашњи детаљи.

Зашто је контрапример погрешан. Контрапример је читање системског времена директно изнутар политике. Тако се тестови везују за сат на рачунару и губе поновљивост.

7.9 Програмски пример и самостални задатак

`RiskBasedAccessPolicy` проверава контекст кроз скуп `IAccessRule` правила. Приступ захтева одговарајућу улогу, уређај, локацију и прихватљив ризик.

Следећи тест издваја један проверљив безбедносни услов. Његов резултат има смисла тек када се објасне улаз, правило и очекивана одлука.

```
[Test]
public void BusinessRoleRule_WhenRoleIsNotResponsible_Denies()
{
    var denial = new BusinessRoleRule(["Operator"])
        .Evaluate(Contexts.Safe with { Role = "Guest" });

    Assert.That(denial?.Code, Is.EqualTo(AccessDecisionCodes.RoleNotResponsible));
}
```

Листинг 7.9.1. Одбијање неприкладне пословне улоге. Извор: AccessRuleTests.cs.

```
cd examples/oib-dotnet/vezba-07-abac-rizik-pregled
dotnet run --project src/Oib.Vezba07.ConsoleUi
dotnet test Oib.Vezba07.sln
```

Листинг 7.9.2. Покретање из репозиторијума наставног портала; потребан је .NET 8 SDK.

Провера	Садржај
Сценарио	Исправна улога користи се са непознатог уређаја или уз висок ризик.
Тестови	AccessRuleTests, RiskBasedAccessPolicyTests, EvaluateAccessHandlerTests
Доказ	AccessDecision даје разлог одлуке; AccessReviewItem одређује рок и власника прегледа.

Табела 7.9.1. Провера и садржај

Самостални задатак

Додати правило радног времена: ограничен ресурс доступан је само радним данима од 8 до 18 часова. Време проследити кроз контекст, а не читати га из система унутар правила.

Питања за самосталну проверу контекстуалне политике

Контекстуална ауторизација мора остати објашњива и када се истом кориснику за исту радњу враћају различите одлуке.

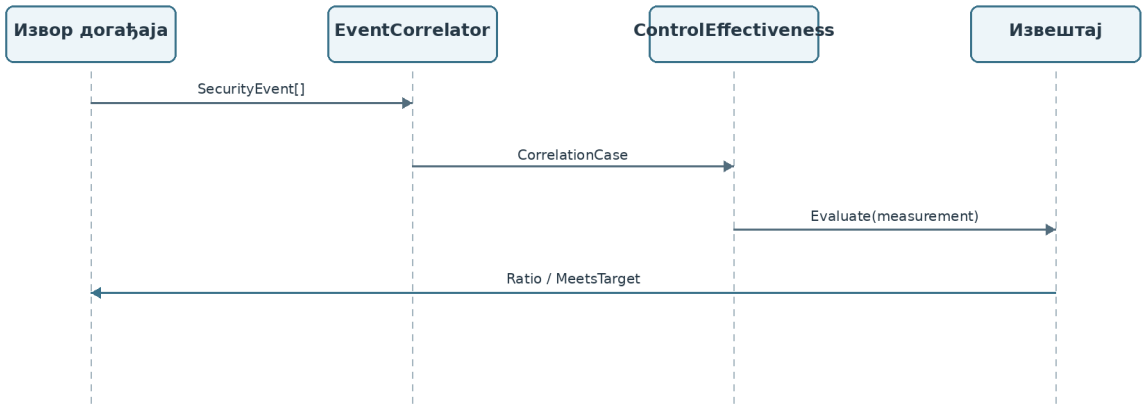
- Који улазни атрибути мењају одлуку и одакле потичу?
- Шта се дешава када је податак о уређају или ризику непознат или застарео?
- Да ли се правило може тестирати без приступа стварном сату и спољном сервису?
- Како ревизија права уклања привилегију која више није оправдана?

Резултат провере треба да покаже не само коначну дозволу него и тачно правило које је пресудно за одлуку.

8 Корелација догађаја, ефективност контрола и учење

Завршна слика безбедности не састоји се од појединачних контрола, већ од њихове везе. Догађаји добијају смисао када се корелишу, контрола добија вредност када постоји свеж доказ да ради, а инцидент постаје прилика да се систем побољша без прикривања грешака.

Мерење ефективности спречава лажни осећај сигурности. Контрола која постоји, али је погрешно подешена, никада тестирана или без власника, не заслужује исто поверење као контрола са проверљивим доказом.



Слика 8.1.1. UML дијаграм секвенце: корелација догађаја и мерење ефективности

Контрола се не сматра делотворном само зато што је имплементирана: потребна су мерења, корелација и објашњење узорка.

Циљ вежбе

Студент треба да образложи одлуку система, податке на којима се она заснива и њен утицај на заштиту података и услуга.

8.1 Контрола има смисао тек уз доказ

Корелација повезује релевантне догађаје у сценарио, док мерење ефективности процењује понашање конкретне контроле над посматраним узорком. Без довољног броја покушаја или одговарајућег циља метрика не оправдава закључак да контрола ради.

- Корелација повезује догађаје кроз време, актере и ресурсе, а не само по једној поруци лога.
- Одговор са већим последицама захтева људску одлуку и јасан траг одобрења.
- Пост-инцидент преглед испитује систем и процес, не тражи најближег кривца.

Питање за дискусију

Изаберите једну контролу и покажите који ризик ублажава, који доказ чува и како бисте приметили да је престала да ради.

8.2 Примена принципа и честе грешке

Добра безбедносна анализа показује узрочну везу: коју имовину штитимо, који сценарио разматрамо, која контрола делује и како знамо да њено понашање још увек одговара намери. Кратак, јасан доказ вреди више од дуге листе функционалности.

Честе грешке које треба препознати

- Закључивање из једног догађаја без ширег контекста.
- Аутоматска акција са великом последицом без заштитне провере.
- Оцена контроле само по томе да ли је имплементирана.

Повезаност са другим целинама

Разматрана одлука повезује идентитет, податке, политике и доказе, а исти поступак анализе примењује се на различите врсте информационих система.

8.3 Пример: контрола постоји, али нема свеж доказ

Систем има правило које ограничава приступ осетљивом ресурсу. Правило је било успешно тестирано пре неколико месеци, али се у међувремену променио модел улога и додат је нови улаз у систем. Без новог доказа тим зна да контрола постоји, али не зна да ли још покрива стварни ризик. Ефективност зато укључује свежину, опсег и резултат провере.

- Корелација повезује низ догађаја у јединствен приказ понашања система.
- Мера ефективности прати да ли контрола има власника, доказ и покривеност релевантног сценарија.
- Аутоматизован одговор може убрзати реакцију, али радња са великом последицом захтева људску потврду и ревизијски траг.

Критеријуми решења

Показати како се израчунава ефективност над валидним подацима и зашто празан узорак не потврђује успех.

8.4 Дизајнерска одлука: корелација пре закључка

Појединачан догађај често има више могућих објашњења. Корелација повезује време, актера, ресурс и ток рада, па тим може да разликује случајну грешку од обрасца који заслужује реакцију. Циљ није прикупити највише података, већ повезати оне податке који мењају одлуку.

- Корелациони идентификатор повезује намеру, обраду и исход једне радње.
- Правила корелације морају бити објашњива, посебно када утичу на приоритет упозорења.
- Аутоматизација предлаже или ограничава одговор када је последица грешке велика.

8.5 Ефективност је више од постојања контроле

Контрола може бити имплементирана, али погрешно подешена, заобиђена новим током или застарела после промене система. Зато се њена ефективност процењује кроз доказ:

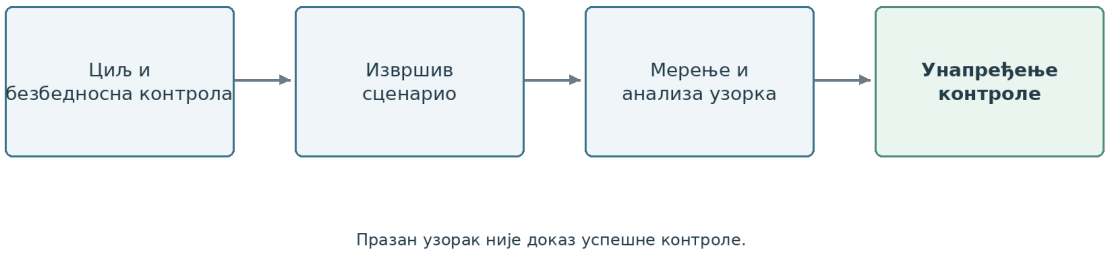
последњи успешан тест, покривени сценарио, власника и способност да се проблем примети ако контрола закаже.

- Доказ се освежава када се промени релевантан ток, улога или граница поверења.
- Мера ефективности не скрива непознато, већ показује где доказа нема довољно.
- Квалитетна анализа повезује ризик, контролу, тест и преостало ограничење у једну јасну причу.

Самостална провера

За изабрани сценарио наведите одлуку система, коришћене податке, очекивани исход и доказ на основу ког други члан тима може да потврди резултат.

8.6 Модел и дијаграм одлуке



Слика 8.6.1. UML дијаграм активности: прикупљање доказа и побољшање контроле

Безбедност напредује кроз повратну спрегу: доказ мења разумевање ризика, а то мења контролу.

Повезати доказ, меру, закључак и одлуку о унапређењу контроле.

8.7 Доказ, покривеност и ефективност

Питање	Слаб одговор	Поуздан одговор
Да ли контрола постоји?	Класа или подешавање је присутно.	Контрола има власника, активну конфигурацију и сврху.
Да ли ради?	Једном је ручно проверена.	Постоји поновљив тест или актуелан оперативни доказ.
Шта покрива?	Претпоставља се да важи свуда.	Познати су сценарио, граница и изузеци.
Како знамо да је ослабила?	Проблем се види тек после инцидента.	Метрика, упозорење или редован преглед откривају одступање.

Табела 8.7.1. Питање и Слаб одговор

Тумачење и примена

Постојање контроле говори да је механизам реализован; ефективност захтева посматрано понашање и праг.

8.8 Анализа захтева и безбедносни доказ

Зрео систем не завршава безбедност имплементацијом контроле. Догађаји и тестови стварају доказ, доказ мења процену ризика, а процена ризика води ка бољој политици или другачијој контроли. Тако се избегава лажни осећај да је једном завршени посао заувек довољан.

29. Догађај или тест открива образац који захтева пажњу.
30. Корелација повезује образац са имовином, актером и контролом.
31. Тим проверава да ли постојећа контрола покрива стварни сценарио.
32. Нова одлука ствара ажурирану политику, тест или оперативну процедуру.

Исход учења

Студент уме да корелира догађаје, провери квалитет мерења и предложи унапређење на основу доказа.

Самостална примена

Применити правило из теме на једној изабраној функционалности: описати субјекат, ресурс, претпоставке и очекивану одлуку, затим припремити негативан тест и запис који објашњава исход.

Проверљив пример: Контрола која постоји, али није проверена

Ситуација. У регистру контрола стоји да се важне радње евидентирају. Међутим, једна контролна мера није ниједном извршена током посматраног периода. Из тога се не сме закључити да је контрола ефективна. Потребно је раздвојити чињеницу да је контрола дефинисана, податак да је примењена и доказ да је постигла очекивани резултат.

Поступак провере. Припремити скуп мерења са нула покушаја, са више успешних покушаја и са више неуспешних покушаја. Израчунати исход и приказати како недовољан узорак утиче на закључак. Потом повезати релевантне догађаје и документовати која тврдња је заиста поткрепљена.

Зашто је контрапример погрешан. Контрапример је једна бројчана оцена „100% ефективно“ када није било прилике да контрола делује. Без броја покушаја, контекста и циља метрика може да доведе до погрешне одлуке.

8.9 Програмски пример и самостални задатак

`EventCorrelator` повезује догађаје у случајеве. `ControlEffectivenessCalculator` пореди удео блокираних покушаја са задатим циљем; без покушаја нема основа за оцену ефективности.

Следећи тест издваја један проверљив безбедносни услов. Његов резултат има смисла тек када се објасне улаз, правило и очекивана одлука.

```
[Test]
public void Evaluate_WhenControlWasNeverExercised_DoesNotClaimItWorks()
{
    var effectiveness = _calculator.Evaluate(new ControlMeasurement("step-up", 0,
0)).Value!;

    using (Assert.EnterMultipleScope())
    {
        Assert.That(effectiveness.Ratio, Is.Zero);
        Assert.That(effectiveness.MeetsTarget, Is.False);
    }
}
```

Листинг 8.9.1. Не оцењивати неиспитану контролу као успешну. Извор: ControlEffectivenessCalculatorTests.cs.

```
cd examples/oib-dotnet/vezba-08-korelacija-efektivnost
dotnet run --project src/Oib.Vezba08.ConsoleUi
dotnet test Oib.Vezba08.sln
```

Листинг 8.9.2. Покретање из репозиторијума наставног портала; потребан је .NET 8 SDK.

Провера	Садржај
Сценарио	Контрола постоји, али не блокира потребан удео ризичних покушаја.
Тестови	EventCorrelatorTests, ControlEffectivenessCalculatorTests, AnalyzeSecurityEventsHandlerTests
Доказ	SecurityAnalysisReport приказује повезане случајеве и резултате мерења.

Табела 8.9.1. Провера и садржај

Самостални задатак

Додати време догађаја и мерити колико је прошло од првог до последњег догађаја у случају. Проширити `CorrelationCase` и написати тест за случај са једним догађајем.

Самостална провера корелације и ефективности

Систем не сме изједначити велики број забележених догађаја са високом ефективношћу контроле. Потребно је проверити квалитет извора, услове корелације и величину узорка.

- Објаснити зашто случај са једним догађајем не мора достићи праг за корелацију.
- Проверити како дуплирани догађаји мењају број покушаја и кориговати правило.
- Раздвојити „контрола није испробана“ од „контрола је испробана и није успела“.
- Навести траг који би независном прегледаоцу омогућио да понови закључак.

Завршни приказ треба да садржи јасан закључак и ограничење закључка, укључујући случај када доказ није довољан за оцену ефективности.

Сажетак: безбедност као следљив, доказив процес

Вежбе примењују исти аналитички след: имовина → претња или злоупотреба → безбедносни захтев → контрола → безбедносни тест → доказ. Поступак се примењује на аутентикацију, привилеговани приступ и анализу путања напада, уз прилагођавање нивоу посматраног система.

Ниво	Шта уводи
P1 — основни	Идентитет, ресурси, класификација, ауторизација, политике и аудит.
P2 — оперативни	вишефакторска аутентикација, сесије, тајне, детекција, инциденти и рањивости.
P3 — напредни	Механизам политика, преглед приступа, ризик, корелација и аутоматизација одговора.

Табела 8.9.2. Ниво и Шта уводи

Одговорност студента

Тим је одговоран за разумевање захтева, образложење безбедносног модела и доказивање ефективности примењене контроле.

Препоручена литература и документација

Литература је намењена продубљивању тема из практикума. Појмове треба повезивати са моделом претњи, безбедносним тестовима и ревизијским записима конкретног информационог система.

- OWASP — Application Security Verification Standard (ASVS) и OWASP Top 10: owasp.org.
- NIST — Digital Identity Guidelines (SP 800-63): pages.nist.gov/800-63-3.
- NIST — Zero Trust Architecture (SP 800-207): csrc.nist.gov.
- Adam Shostack — Threat Modeling: Designing for Security.
- NUnit документација: docs.nunit.org; Moq документација: github.com/devlooped/moq.
- Microsoft Learn — ASP.NET Core Identity, ауторизација и безбедна конфигурација: learn.microsoft.com/aspnet/core/security.
- MITRE ATT&CK — референтни оквир за тактике и технике, не упутство за извођење напада: attack.mitre.org.

Напомена о алатима

Конкретни безбедносни алати (SIEM, IDS, PKI производи) мењају се брже од основних принципа. Симулатори коришћени у практикуму опонашају њихово понашање ради вежбе — не представљају се као реални производи.